

Introduction to Artificial Intelligence

Lecture 3: Games and Adversarial search

Prof. Gilles Louppe

g.louppe@uliege.be



Today

How to act rationally in a **multi-agent** environment? How to anticipate and respond to the **arbitrary behavior** of other agents?

- Adversarial search
 - Minimax
 - α - β pruning
 - H-Minimax
 - Expectiminimax
 - Monte Carlo Tree Search
- Modeling assumptions
- State-of-the-art agents.

Games

Formal definition

A **game** is defined by

- the **states** $s \in \mathcal{S}$ and the **initial state** $s_1 \in \mathcal{S}$;
- the **actions** $\text{actions}(s) \subseteq \mathcal{A}$ available in state s ;
- the **transition model** $s' = \text{result}(s, a)$, the state reached by taking a in s ;
- a function $\text{player}(s) \in \{1, \dots, N\}$ that says whose turn it is in state s ;
- the **terminal states** $T \subseteq \mathcal{S}$, tested by $\text{terminal-test}(s)$, i.e. $s \in T$;
- a **utility** $\text{utility}(s, p)$, the payoff of player p when the game ends in $s \in T$, e.g. 1 , 0 or $\frac{1}{2}$ for a win, a loss or a draw.

Together, s_1 , **actions** and **result** define the **game tree**, whose nodes are states and whose edges are actions.

Strategies

Even in a deterministic and fully observable environment, the opponents may act **arbitrarily**: the next state depends on their choices, which we cannot predict.

A fixed sequence of actions $a_{1:T}$, the solution of a search problem, is then of no use, since the opponent may lead the game anywhere. A **solution** is instead a **strategy**, or **policy**, $\pi_p : \mathcal{S} \rightarrow \mathcal{A}$ for player p : a move for every state the opponent can lead to.

The bar scene, formally

Let the N friends choose in turn whom to approach, B (the blonde) or F (one of her friends):

- a state s is the sequence of choices made so far, and $s_1 = ()$;
- $\text{actions}(s) = \{B, F\}$ and $\text{result}(s, a)$ appends a to s ;
- $\text{player}(s) = |s| + 1$, the friend whose turn it is;
- the terminal states T are the sequences of length N , when everyone has chosen;
- $\text{utility}(s, p)$ is 2 if p is alone to choose B, 0 if others chose B too, and 1 if p chose F.

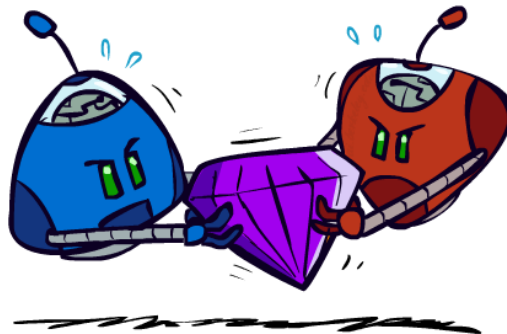
Nash evaluates **outcomes**, not moves: everyone for the blonde gives 0 to every player, nobody for the blonde gives 1 . What each friend should do depends on what the others do, so the answer is a **strategy** rather than a sequence of moves.

Zero-sum games

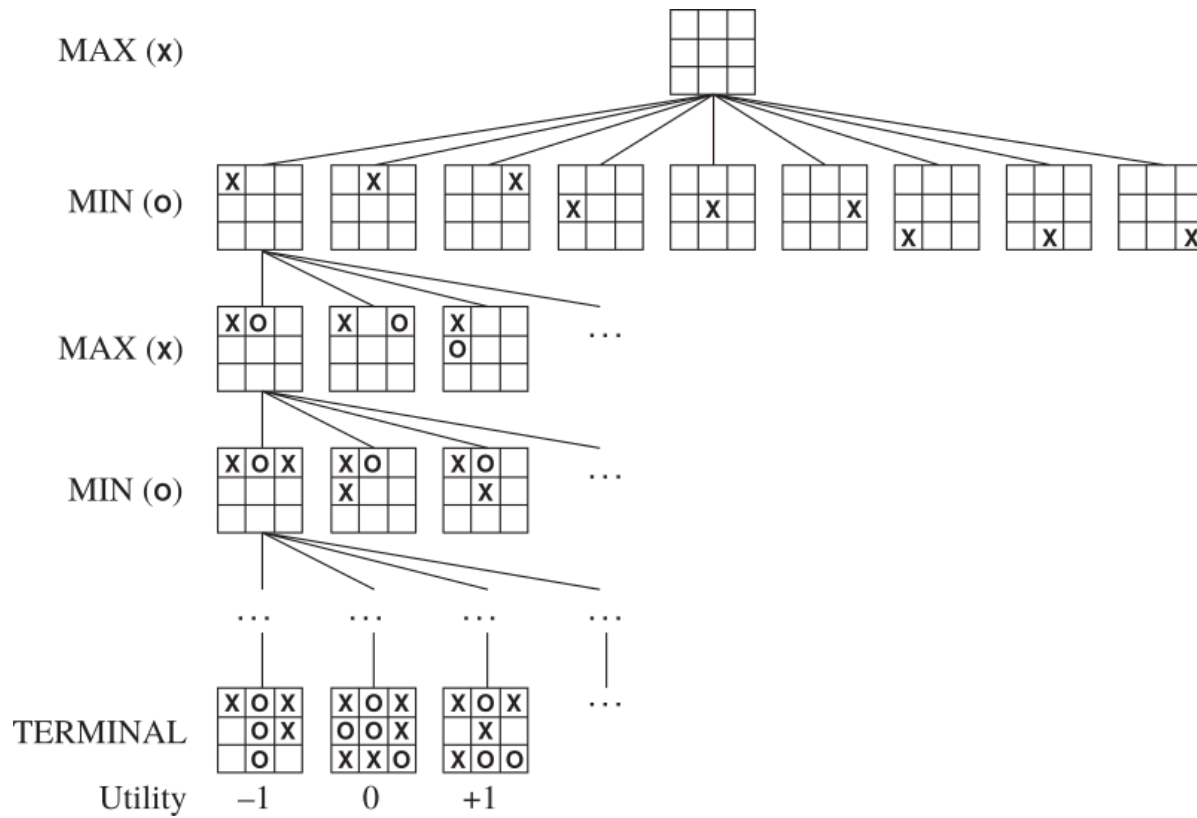
A game is **zero-sum** if the utilities of the players sum to the same constant c in every terminal state:

$$\sum_{p=1}^N \text{utility}(s, p) = c \quad \text{for all } s \in T.$$

- e.g., in chess, $c = 1$: $1 + 0, 0 + 1$ or $\frac{1}{2} + \frac{1}{2}$.
- With two players, $\text{utility}(s, 2) = c - \text{utility}(s, 1)$: a **single utility** describes the game, which player 1 **maximizes** and player 2 **minimizes**.
- This is **strict competition**: what one player gains, the other loses.



Tic-Tac-Toe game tree



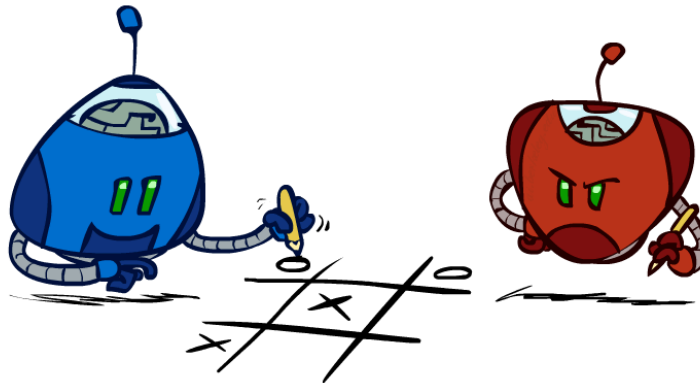
What is an optimal strategy (or perfect play)? How do we find it?

Minimax

Assumptions

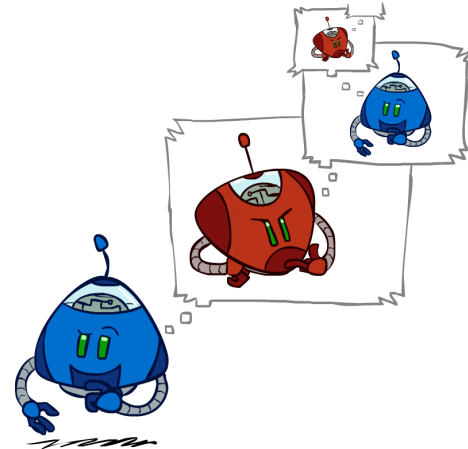
Games vary along three axes: **chance**, **information** and the **number of players**. We start with the simplest case of each.

- We assume a **deterministic, turn-taking, two-player zero-sum game** with **perfect information**.
 - e.g., Tic-Tac-Toe, Chess, Checkers, Go, etc.
- We will call our two players MAX and MIN. MAX moves first.



Adversarial search

- In a search problem, the optimal solution is a sequence of actions leading to a goal state.
 - i.e., a terminal state where MAX wins.
- In a game, the opponent (MIN) may react **arbitrarily** to a move.
- Therefore, a player (MAX) must define a contingent **strategy** π_{MAX} which specifies
 - its moves in the initial state,
 - its moves in the states resulting from every possible response by MIN,
 - its moves in the states resulting from every possible response by MIN in those states, ...



Minimax

The **minimax value** $\text{minimax}(s)$ is the largest achievable payoff (for MAX) from state s , assuming an **optimal adversary** (MIN),

$$\text{minimax}(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a \text{minimax}(\text{result}(s, a)) & \text{if MAX plays} \\ \min_a \text{minimax}(\text{result}(s, a)) & \text{if MIN plays,} \end{cases}$$

where the **max** and **min** are over $a \in \text{actions}(s)$.

The **optimal** next move (for MAX) maximizes the minimax value of the resulting state, $\pi^*(s) = \arg \max_a \text{minimax}(\text{result}(s, a))$.

- Assuming that MIN is an optimal adversary that maximizes the **worst-case outcome** for MAX.
- This is equivalent to not making an assumption about the strength of the opponent.

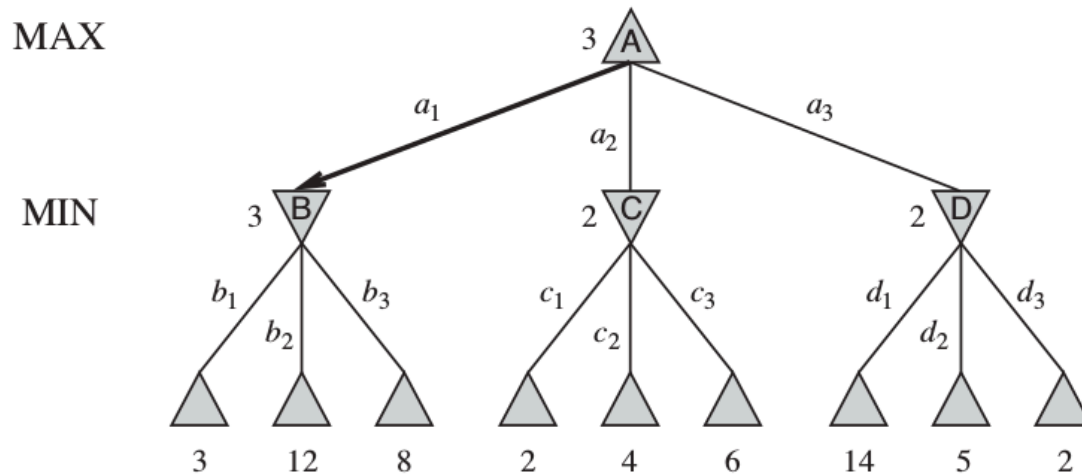


Figure 5.2 A two-ply game tree. The $\triangle$ nodes are “MAX nodes,” in which it is MAX’s turn to move, and the ∇ nodes are “MIN nodes.” The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX’s best move at the root is a_1 , because it leads to the state with the highest minimax value, and MIN’s best reply is b_1 , because it leads to the state with the lowest minimax value.

Properties of Minimax

- **Completeness:**

- Yes, if tree is finite.

- **Optimality:**

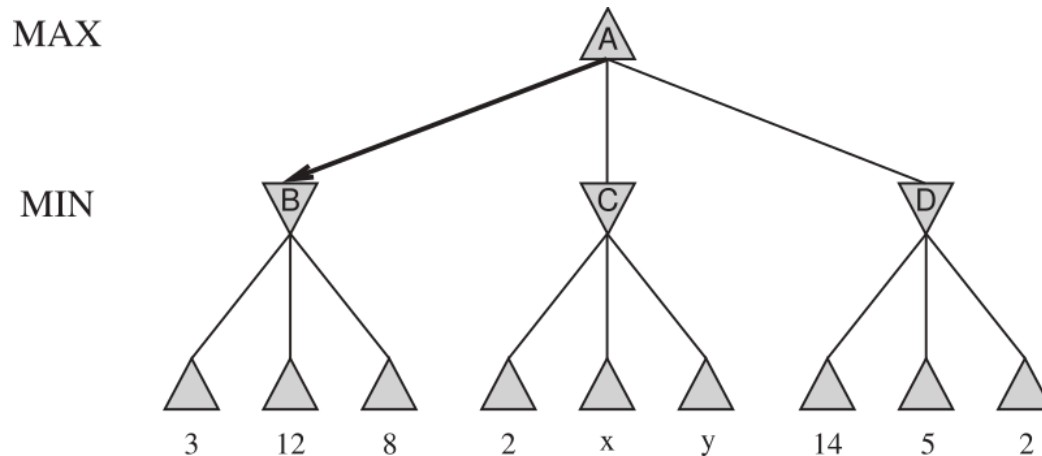
- Yes, if MIN is an optimal opponent.
- What if MIN is suboptimal?
 - Show that MAX will do even better.
- What if MIN is suboptimal and predictable?
 - Other strategies might do better than Minimax. However they may do worse on an optimal opponent.

Minimax efficiency

- Assume $\text{minimax}(s)$ is implemented using its recursive definition.
- How efficient is minimax?
 - Time complexity: same as DFS, i.e., $O(b^m)$.
 - Space complexity:
 - $O(bm)$, if all actions are generated at once, or
 - $O(m)$, if actions are generated one at a time.

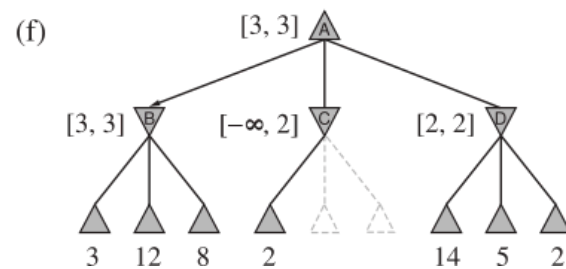
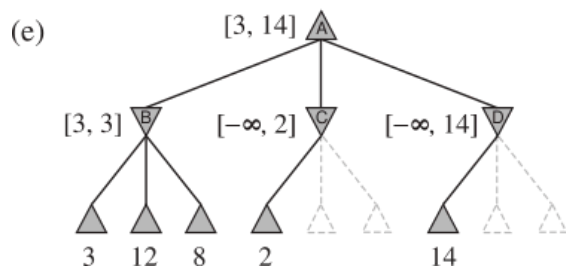
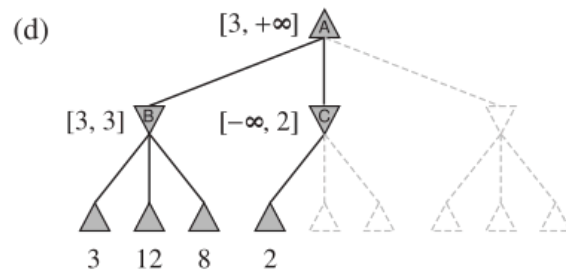
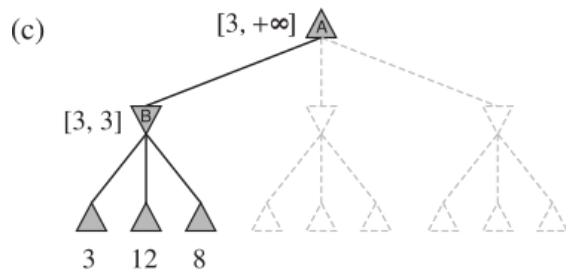
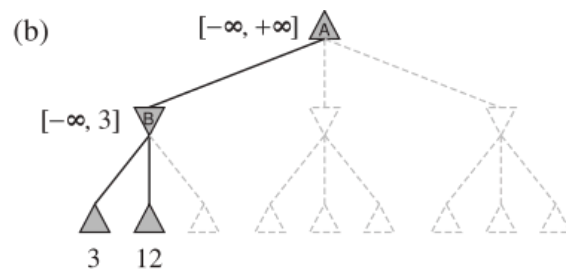
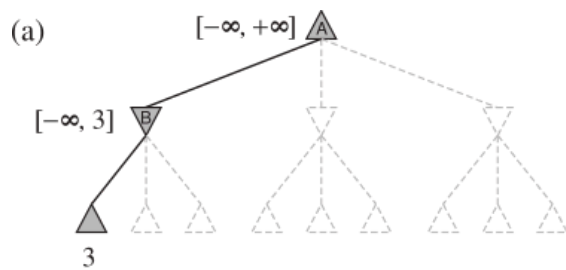
Do we need to explore the whole game tree?

Pruning



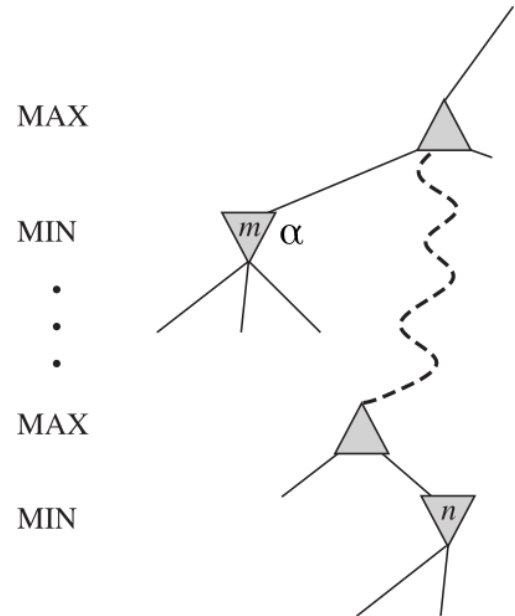
$$\begin{aligned}\text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\ &= \max(3, \min(2, x, y), 2) \\ &= \max(3, z, 2) \quad \text{where } z = \min(2, x, y) \leq 2 \\ &= 3.\end{aligned}$$

Therefore, it is possible to compute the **correct** minimax decision **without looking at every node** in the tree.



We want to compute $v = \text{minimax}(n)$, for $\text{player}(n) = \text{MIN}$.

- We loop over n 's children.
- The minimax values are being computed one at a time and v is updated iteratively.
- Let α be the best value (i.e., the highest) at any choice point along the path for MAX.
- If v becomes lower than α , then n **will never be reached** in actual play.
- Therefore, we can **stop iterating** over the remaining n 's other children.



Similarly, β is defined as the best value (i.e., lowest) at any choice point along the path for MIN. We can halt the expansion of a MAX node as soon as v becomes larger than β .

α - β pruning

- Updates the values of α and β as the path is expanded.
- Prune the remaining branches (i.e., terminate the recursive calls) as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.

α - β search

function ALPHA-BETA-SEARCH($state$) **returns** an action

$v \leftarrow \text{MAX-VALUE}(state, -\infty, +\infty)$

return the *action* in ACTIONS($state$) with value v

function MAX-VALUE($state, \alpha, \beta$) **returns** a utility value

if TERMINAL-TEST($state$) **then return** UTILITY($state$)

$v \leftarrow -\infty$

for each a in ACTIONS($state$) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(state, a), \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

function MIN-VALUE($state, \alpha, \beta$) **returns** a utility value

if TERMINAL-TEST($state$) **then return** UTILITY($state$)

$v \leftarrow +\infty$

for each a in ACTIONS($state$) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(state, a), \alpha, \beta))$

if $v \leq \alpha$ **then return** v

$\beta \leftarrow \text{MIN}(\beta, v)$

return v

Properties of α - β search

- Pruning has **no effect** on the minimax values. Therefore, **completeness** and **optimality** are preserved from Minimax.
- Time complexity:
 - The effectiveness depends on the order in which the states are examined.
 - If states could be examined in **perfect order**, then α - β search examines only $O(b^{m/2})$ nodes to pick the best move, vs. $O(b^m)$ for minimax.
 - α - β can solve a tree twice as deep as minimax can in the same amount of time.
 - Equivalent to an effective branching factor $\sqrt{b}$.
- Space complexity: $O(m)$, as for Minimax.

Game tree size



Chess:

- $b \approx 35$ (approximate average branching factor)
- $d \approx 100$ (depth of a game tree for typical games)
- $b^d \approx 35^{100} \approx 10^{154}$.
- For α - β search and perfect ordering, we get $b^{d/2} \approx 35^{50} = 10^{77}$.

Finding the exact solution with Minimax remains **intractable**.

Transposition table

- Repeated states occur frequently because of **transpositions**: distinct permutations of the move sequence end in a same position.
- Similarly to the **closed** set in graph search (Lecture 2), it is worth storing the evaluation of a state such that further occurrences of the state do not have to be recomputed.

What data structure should be used to efficiently store and look-up values of positions?

Imperfect real-time decisions

- Under **time constraints**, searching for the exact solution is not feasible in most realistic games.
- Solution: cut the search earlier.
 - Replace the **utility**(s) function with a heuristic **evaluation function** $\text{eval}(s)$ that estimates the state utility.
 - Replace the terminal test by a **cutoff test** $\text{cutoff-test}(s, d)$ that decides when to stop expanding a state s at depth d .

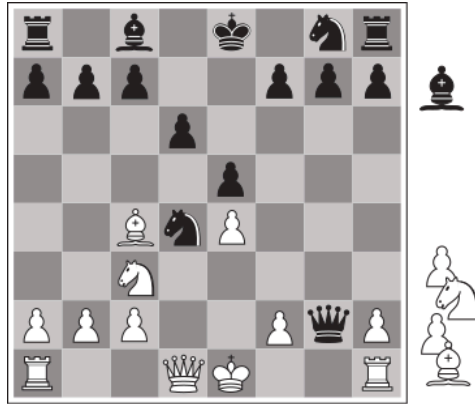
$$\text{h-minimax}(s, d) = \begin{cases} \text{eval}(s) & \text{if cutoff} \\ \max_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MAX plays} \\ \min_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MIN plays.} \end{cases}$$

Can α - β search be adapted to implement H-Minimax?

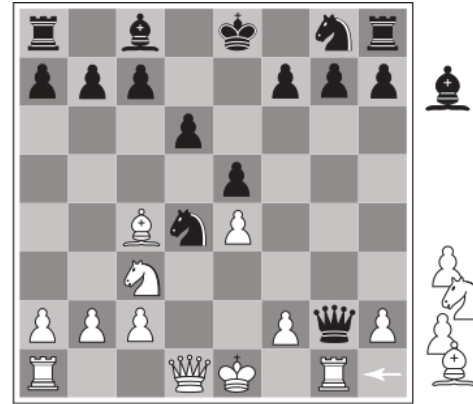
Evaluation functions

- An evaluation function `eval(s)` returns an **estimate** of the expected utility of the game from a given position *s*.
- The computation **must be short** (that is the whole point to search faster).
- Ideally, the evaluation should **order** states in the same way as in Minimax.
 - The evaluation values may be different from the true minimax values, as long as order is preserved.
- In non-terminal states, the evaluation function should be strongly **correlated** with the actual chances of winning.

Quiescence



(a) White to move



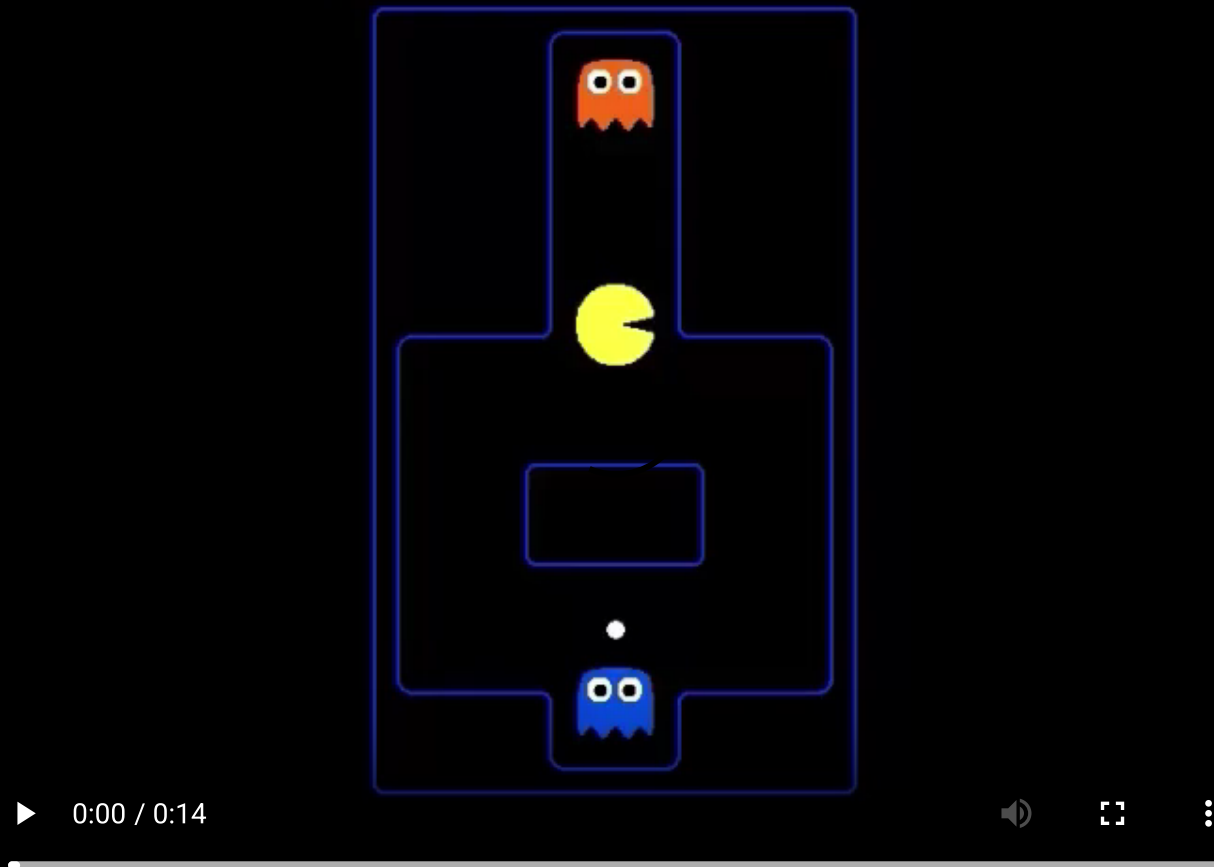
(b) White to move

- These states only differ in the position of the rook at lower right.
- However, Black has advantage in (a), but not in (b).
- If the search stops in (b), Black will not see that White's next move is to capture its Queen, gaining advantage.
- Cutoff should only be applied to positions that are **quiescent**.
 - i.e., states that are unlikely to exhibit wild swings in value in the near future.

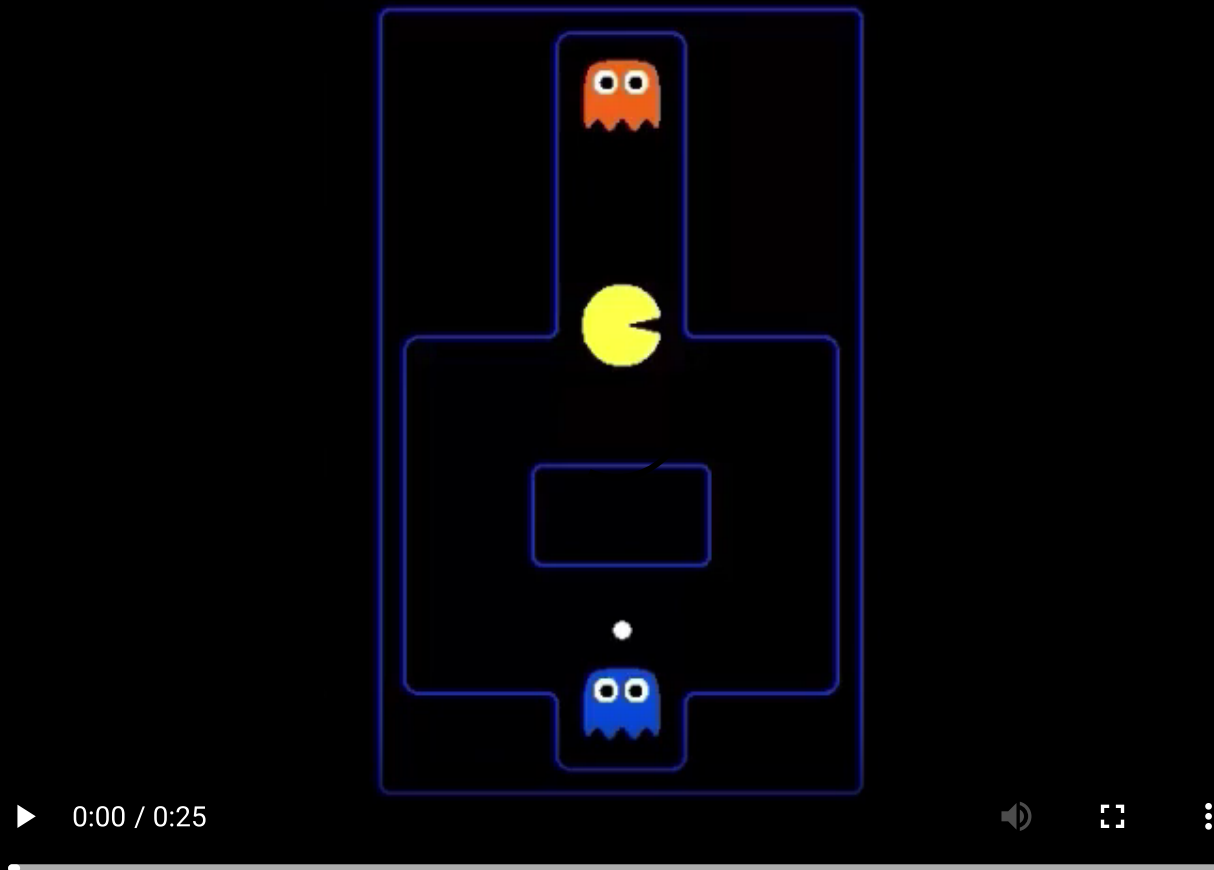
The horizon effect

Evaluations functions are **always imperfect**.

- If not looked deep enough, **bad moves** may appear as **good moves** (as estimated by the evaluation function) because their consequences are hidden beyond the search horizon.
 - and vice-versa!
- Often, the deeper in the tree the evaluation function is buried, the less the quality of the evaluation function matters.



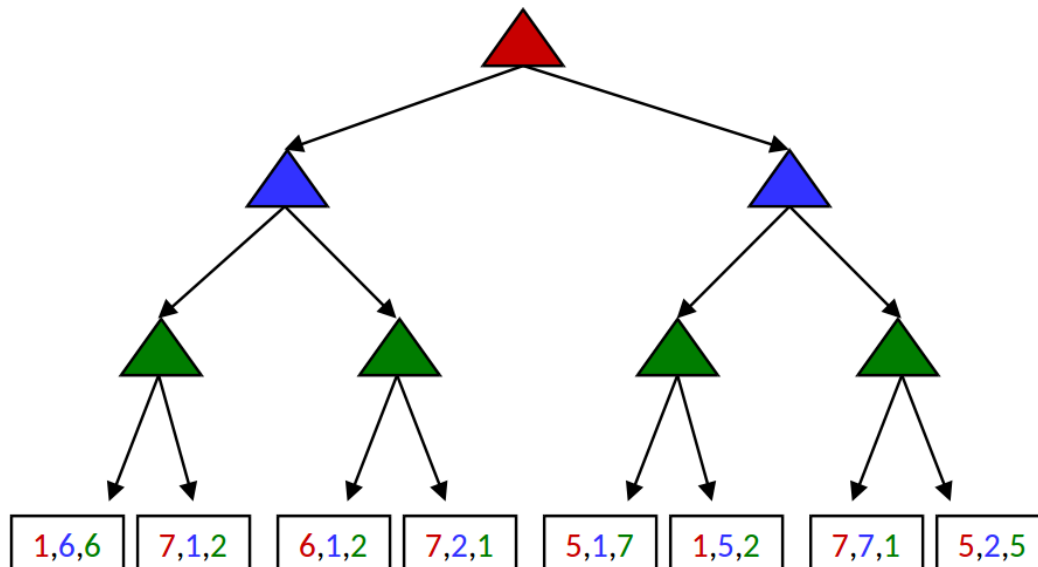
Cutoff at depth 2, evaluation = the closer to the dot, the better.



Cutoff at depth 10, evaluation = the closer to the dot, the better.

Multi-agent games

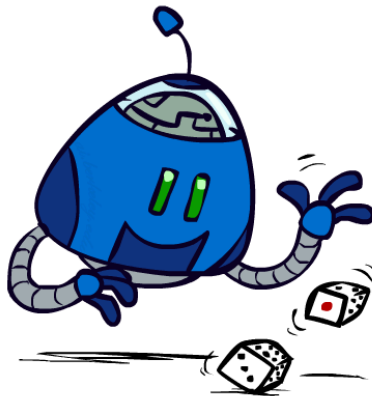
- What if the game is not zero-sum, or has **multiple players**?
- Generalization of Minimax:
 - Terminal states are labeled with utility **tuples** (1 value per player).
 - Intermediate states are also labeled with utility tuples.
 - Each player maximizes its own component.
 - May give rise to cooperation and competition dynamically.



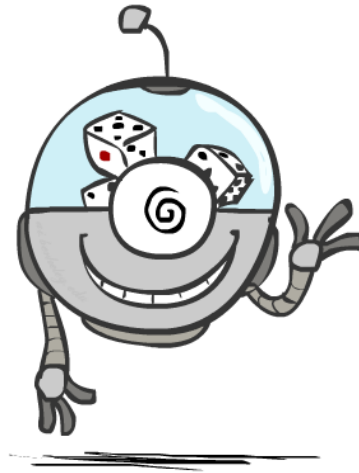
Stochastic games

Stochastic games

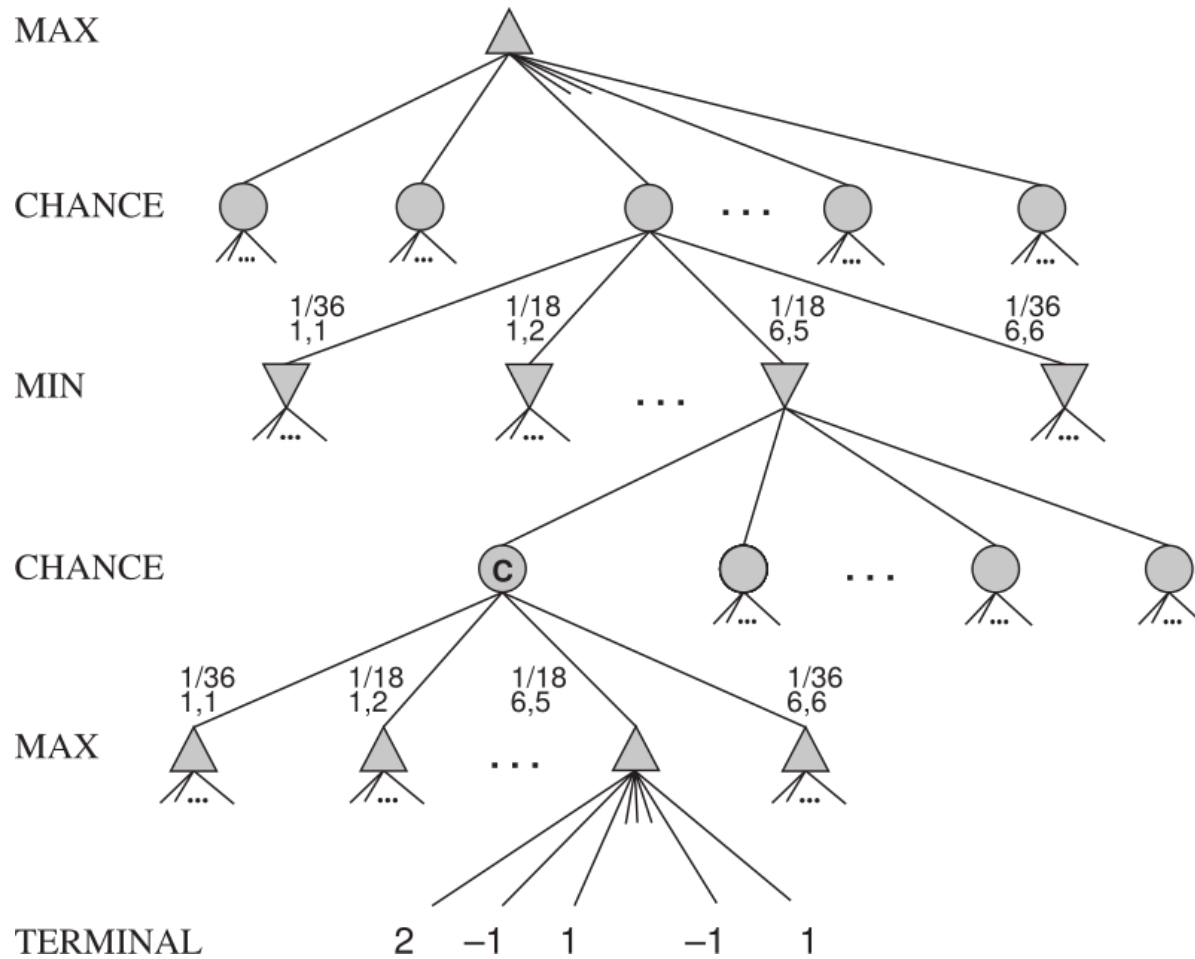
- In real life, many unpredictable external events can put us into unforeseen situations.
- Games that mirror this unpredictability are called **stochastic games**. They include a random element, such as:
 - explicit randomness: rolling a dice;
 - actions may fail: when moving a robot, wheels might slip.



- In a game tree, this random element can be **modeled** with **chance nodes** that map a state-action pair to the set of possible outcomes, along with their respective **probability**.
- This is equivalent to considering the environment as an extra **random agent** player that moves after each of the other players.



Stochastic game tree



Expectiminimax

- Because of the uncertainty in the action outcomes, states no longer have a **definite minimax** value.
- However, we can calculate the **expected** value of a state under optimal play by the opponent.
 - i.e., the average over all possible outcomes of the chance nodes.
 - **minimax** values correspond instead to the worst-case outcome.

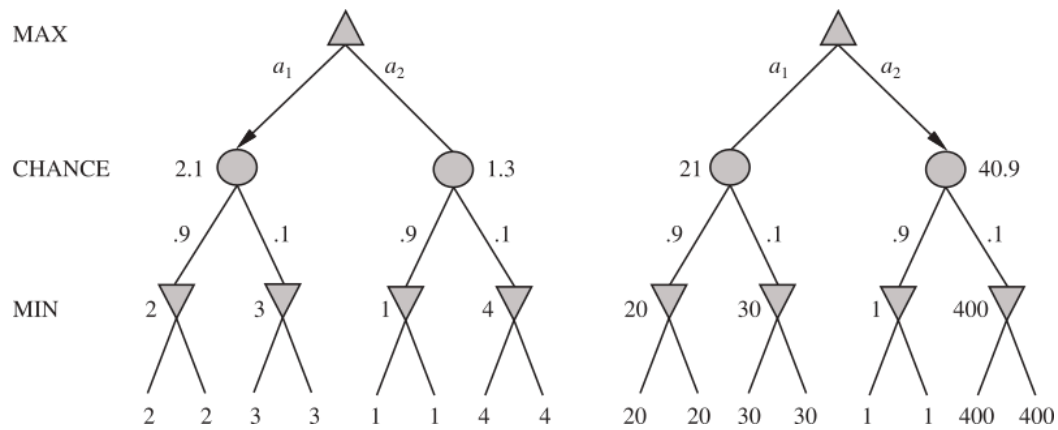
Writing $v(s) = \text{expectiminimax}(s)$ for short,

$$v(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a v(\text{result}(s, a)) & \text{if MAX plays} \\ \min_a v(\text{result}(s, a)) & \text{if MIN plays} \\ \sum_r P(r) v(\text{result}(s, r)) & \text{if CHANCE plays.} \end{cases}$$

Does taking the rational move mean the agent will be successful?

Evaluation functions

- As for $\text{minimax}(n)$, the value of $\text{expectiminimax}(n)$ may be approximated by stopping the recursion early and using an evaluation function.
- However, to obtain correct move, the evaluation function should be a **positive linear transformation** of the expected utility of the state.
 - It is not enough for the evaluation function to just be order-preserving.
- If we assume bounds on the utility function, α - β search can be adapted to stochastic games.



An order-preserving transformation on leaf values changes the best move.

Monte Carlo tree search

Monte Carlo Tree Search

Random playout evaluation

- To evaluate a state, have the algorithm play **against itself** using **random moves**, thousands of times.
- The sequence of random moves is called a **random playout**.
- Use the proportion of wins as the state evaluation.
- This strategy does **not require domain knowledge!**
 - The game engine is all that is needed.

Monte Carlo Tree Search

The focus of MCTS is the analysis of the most promising moves, as incrementally evaluated with random playouts.

Each node n in the current search tree maintains two values:

- the number of wins $Q(n, p)$ of player p for all playouts that passed through n ;
- the number $N(n)$ of times n has been visited.

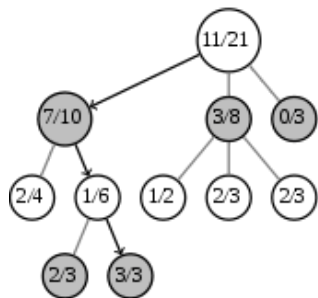
Each round of the search does four things: **selection** down to a node that is not fully expanded, **expansion** of one child, **simulation** of a random playout from it, and **backpropagation** of the result along the path back to the root.

```
function MCTS(state, budget) returns an action
  root  $\leftarrow$  MAKE-NODE(state)
  while time remains in budget do
    n  $\leftarrow$  SELECT(root)
    n'  $\leftarrow$  EXPAND(n)
    reward  $\leftarrow$  SIMULATE(n')
    BACKPROPAGATE(n', reward)
  end
  return the action leading to the child of root with the largest N
```

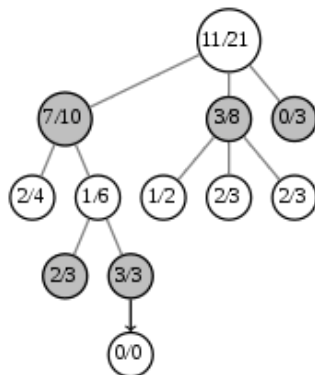
```
function SELECT(n) returns a node to expand
  while n is fully expanded and not terminal do
    n  $\leftarrow$  the child of n maximising UCB1
  end
  return n
```

Rounds are repeated for as long as the time budget allows. The move played is the one leading to the most visited child of the root.

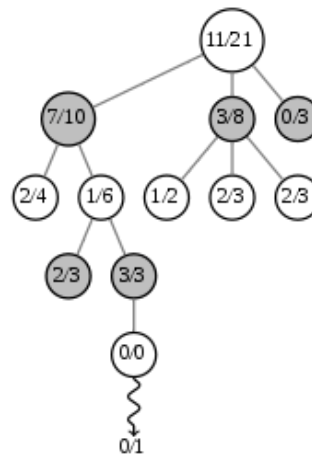
Selection



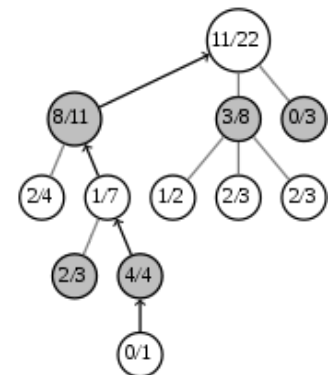
Expansion



Simulation



Backpropagation



Exploration and exploitation

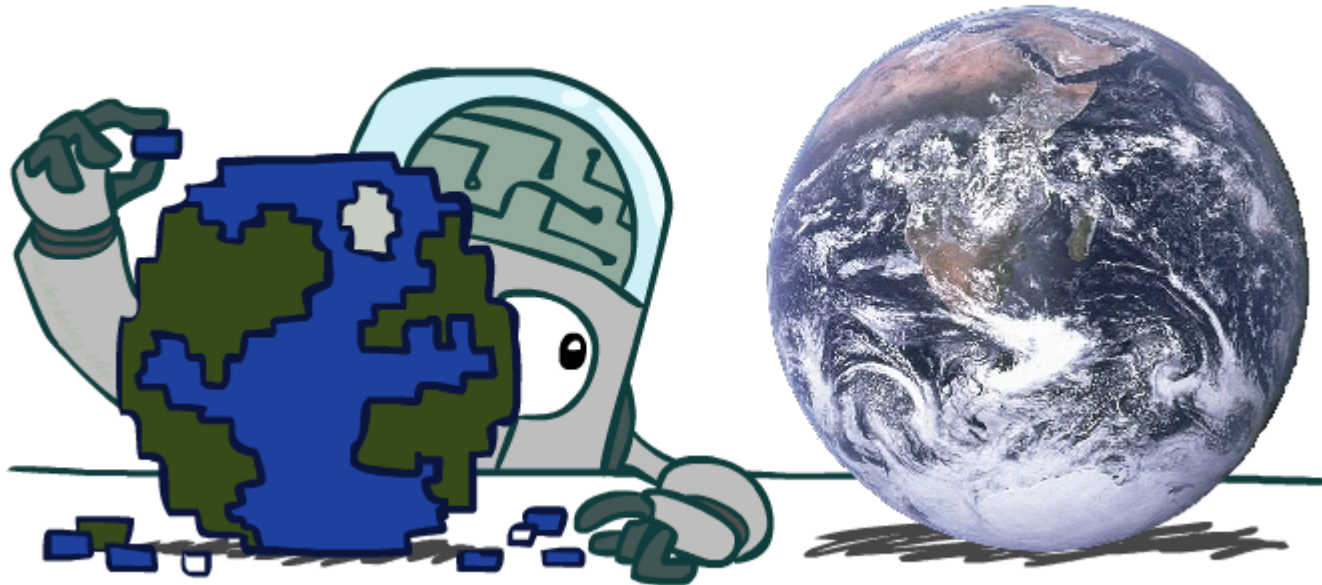
Given a limited budget of random playouts, the efficiency of MCTS critically depends on the choice of the nodes made during the **selection** step.

During the traversal of the branch in the selection step, the UCB1 policy picks the child node n' of n that maximizes

$$\frac{Q(n', p)}{N(n')} + c \sqrt{\frac{\log N(n)}{N(n')}}.$$

- The first term encourages the **exploitation** of higher-reward nodes.
- The second term encourages the **exploration** of less-visited nodes.
- The constant $c > 0$ controls the trade-off between exploitation and exploration.

Modeling assumptions

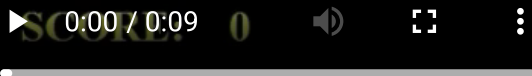
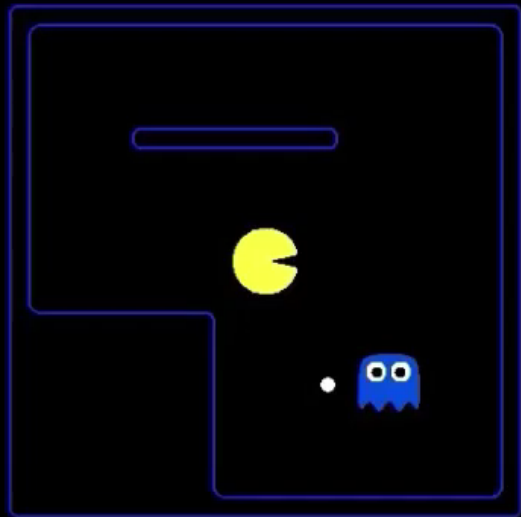


What if our assumptions are incorrect?

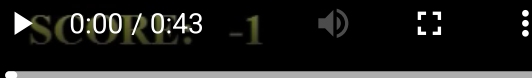
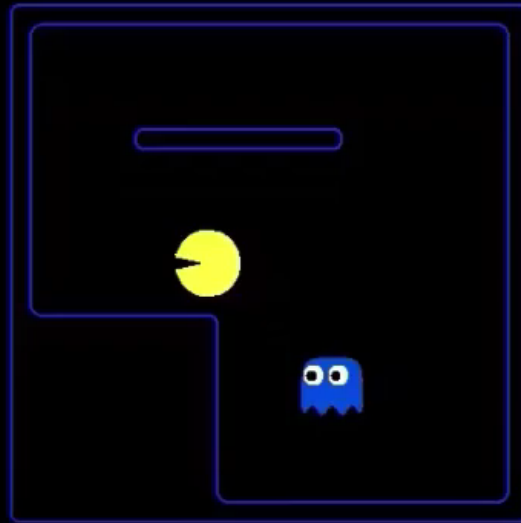
Setup

- P_1 : Pacman uses depth 4 search with an evaluation function that avoids trouble, while assuming that the ghost follows P_2 .
- P_2 : Ghost uses depth 2 search with an evaluation function that seeks Pacman, while assuming that Pacman follows P_1 .
- P_3 : Pacman uses depth 4 search with an evaluation function that avoids trouble, while assuming that the ghost follows P_4 .
- P_4 : Ghost makes random moves.

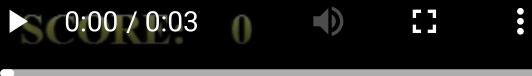
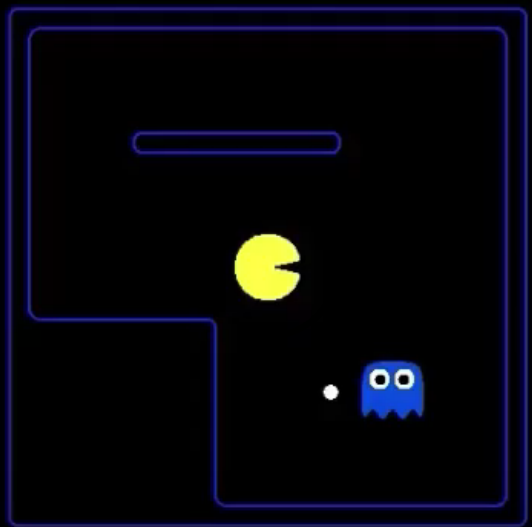




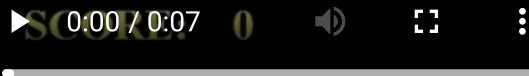
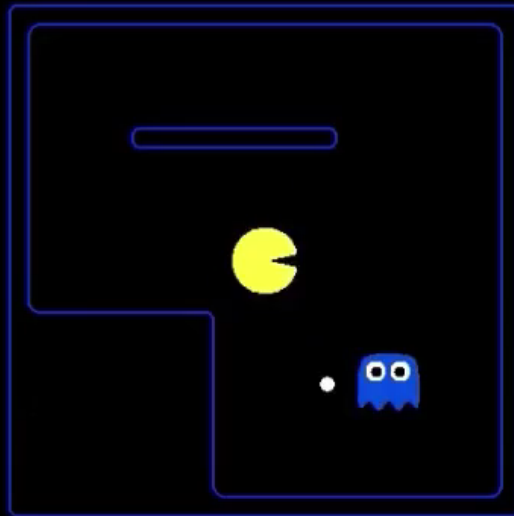
Minimax Pacman (P_1) vs. Adversarial ghost (P_2)



Minimax Pacman (P_1) vs. Random ghost (P_4)



Expectiminimax Pacman (P_3) vs. Random ghost (P_4)



Expectiminimax Pacman (P_3) vs. Adversarial ghost (P_2)

State-of-the-art game programs

Perfect information

- **1997, Deep Blue** defeats Garry Kasparov at chess: 2×10^8 positions per second, a sophisticated evaluation function, and search extended up to 40 plies on some lines.
- **2007, checkers is solved:** 10^{14} calculations over 18 years prove that perfect play by both sides is a draw.
- **2016, AlphaGo** beats Lee Sedol 4-1, then Ke Jie in 2017, by combining Monte Carlo tree search with deep learning trained on human and self-play games.

Chess and Go remain **unsolved**: only their best play is now superhuman.

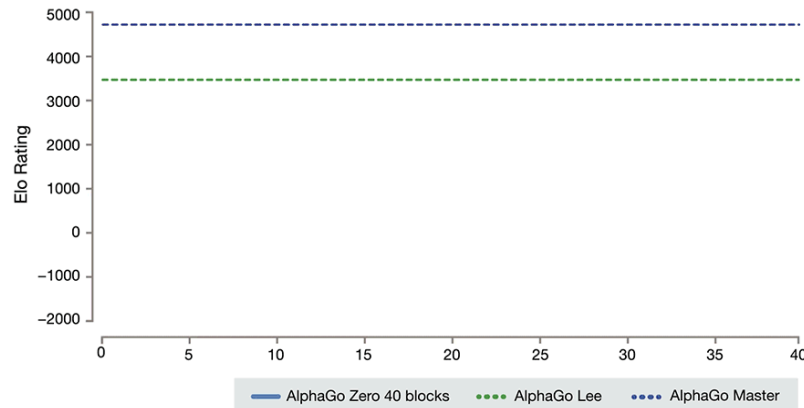




Press coverage for the victory of AlphaGo against Lee Sedol.

Learning to play, from less and less

- **2017, AlphaGo Zero** reaches the same level from **self-play only**, with no human games.
- **2018, AlphaZero** applies the same recipe to chess, shogi and Go, given only the rules.
- **2020, MuZero** drops the rules too: it **learns a model** of the game and plans with it, mastering Go, chess, shogi and Atari.



Imperfect information

The assumption of **perfect information** we made at the start is what these games drop:

- **2017-2019, Libratus** then **Pluribus** beat professionals at no-limit poker, the latter with **six players** at the table, where no notion of optimal play against all opponents exists.
- **2022, DeepNash** reaches expert level at **Stratego**, where each side hides its pieces.
- **2022, Cicero** plays **Diplomacy** at human level, combining planning with **negotiation in natural language**.

What does an optimal strategy even mean when the opponents may cooperate, and lie?

Summary

- A **game** adds **player**(s), **terminal states** $T \subseteq \mathcal{S}$ and a **utility** $\text{utility}(s, p)$ to the search problem of lecture 2. Its solution is a **strategy** $\pi_p : \mathcal{S} \rightarrow \mathcal{A}$, not a sequence $a_{1:T}$.
- **Minimax** backs the utilities up the tree, $\max_a$ for MAX and $\min_a$ for MIN, in $O(b^m)$ time and $O(bm)$ space.
- α - β **pruning** returns the **same move** while examining $O(b^{m/2})$ nodes at best: twice the depth in the same time.
- **H-minimax** cuts the search at depth d and replaces the utility by an **evaluation function**, at the price of the **horizon effect**.
- **Expectiminimax** adds **chance nodes**, worth $\sum_r P(r) v(\text{result}(s, r))$, and **Monte Carlo tree search** samples **random playouts** instead of evaluating.
- Each is optimal only **relative to a model** of the opponent. Assuming the **wrong** one is not playing well.

